Used Car Price Prediction: Classification and Regression

Course: Machine Learning
Environment: Google Colab / Python
Dataset: UCI Automobile Data Set

Project overview

This notebook demonstrates two supervised machine learning tasks using one automobile dataset:

1. **Classification:** Predict whether a car belongs to the Budget, Mid-range, or Premium price category.
2. **Regression:** Predict a car's numerical price.

The dataset contains automobile specifications and a `price` target. For classification, we create price categories from the price column, then **remove the original price from the input features** to prevent target leakage.

> **Dataset note:** The UCI Automobile dataset describes automobile specifications and is not a modern used-car listings dataset. The project title is used-car-price-themed, but conclusions should be limited to this dataset and its small sample size.

1. Import libraries

We use Pandas and NumPy for data handling, Matplotlib and Seaborn for visualization, and Scikit-learn for preprocessing, model training, and evaluation.

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix,
    mean_absolute_error, mean_squared_error, r2_score
)

RANDOM_STATE = 42
pd.set_option("display.max_columns", 100)

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

2. Load the dataset

The UCI raw file contains 205 records and 26 attributes. The source file does not include a header row, so we provide the column names. Question marks are converted to missing values (`NaN`).

```
url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/autos/imports-85.data"
)

columns = [
    "symboling", "normalized-losses", "make",
```

```
    "fuel-type", "aspiration", "num-of-doors",
    "body-style", "drive-wheels", "engine-location",
    "wheel-base", "length", "width", "height",
    "curb-weight", "engine-type", "num-of-cylinders",
    "engine-size", "fuel-system", "bore", "stroke",
    "compression-ratio", "horsepower", "peak-rpm",
    "city-mpg", "highway-mpg", "price"
]

df = pd.read_csv(url, names=columns, na_values="?")

# Convert numeric-looking columns to numeric values.
numeric_candidates = [
    "symboling", "normalized-losses", "wheel-base", "length", "width",
    "height", "curb-weight", "engine-size", "bore", "stroke",
    "compression-ratio", "horsepower", "peak-rpm", "city-mpg",
    "highway-mpg", "price"
]
for col in numeric_candidates:
    df[col] = pd.to_numeric(df[col], errors="coerce")

print("Dataset loaded successfully.")
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset loaded successfully.
Dataset shape: (205, 26)
```

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	length	width	height	curb-weight	e
0	3	NaN	alfa-romero	gas	std	two	convertible	rwd	front	88.6	168.8	64.1	48.8	2548	
1	3	NaN	alfa-romero	gas	std	two	convertible	rwd	front	88.6	168.8	64.1	48.8	2548	
2	1	NaN	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	171.2	65.5	52.4	2823	
3	2	164.0	audi	gas	std	four	sedan	fwd	front	99.8	176.6	66.2	54.3	2337	
4	2	164.0	audi	gas	std	four	sedan	4wd	front	99.4	176.6	66.4	54.3	2824	

3. Understand and inspect the data

Before modeling, inspect data types, missing values, duplicate rows, and basic descriptive statistics. This is part of exploratory data analysis (EDA).

```
print("Dataset information:")
df.info()

print("\nMissing values by column:")
display(df.isna().sum().sort_values(ascending=False).to_frame("missing_count"))

print("\nDuplicate rows:", df.duplicated().sum())

print("\nNumeric summary:")
display(df.describe().T)
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 26 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   symboling          205 non-null    int64
 1   normalized-losses  164 non-null    float64
 2   make               205 non-null    object
 3   fuel-type          205 non-null    object
 4   aspiration         205 non-null    object
 5   num-of-doors       203 non-null    object
 6   body-style         205 non-null    object
 7   drive-wheels       205 non-null    object
 8   engine-location    205 non-null    object
 9   wheel-base         205 non-null    float64
 10  length             205 non-null    float64
 11  width              205 non-null    float64
 12  height             205 non-null    float64
 13  curb-weight        205 non-null    int64
 14  engine-type        205 non-null    object
 15  num-of-cylinders   205 non-null    object
 16  engine-size        205 non-null    int64
 17  fuel-system        205 non-null    object
 18  bore               201 non-null    float64
 19  stroke             201 non-null    float64
 20  compression-ratio  205 non-null    float64
 21  horsepower         203 non-null    float64
 22  peak-rpm           203 non-null    float64
 23  city-mpg           205 non-null    int64
 24  highway-mpg        205 non-null    int64
 25  price              201 non-null    float64
dtypes: float64(11), int64(5), object(10)
memory usage: 41.8+ KB

Missing values by column:
```

4. Data cleaning

	missing_count
normalized-losses	41
stroke	4
price	4
fuel-type	0
make	0
wheel-base	0
engine-location	0
drive-wheels	0
body-style	0
aspiration	0
num-of-cylinders	0
engine-type	0
curb-weight	0
compression-ratio	0
city-mpg	0
highway-mpg	0

Rows without a target price cannot be used for supervised training, so we remove those rows. We keep other missing feature values for now; the preprocessing pipelines will impute them using statistics learned from the training split. This avoids fitting imputers on test data.

```
df = df.dropna(subset=["price"]).copy()
df["price"] = pd.to_numeric(df["price"], errors="coerce")
df = df.dropna(subset=["price"])
df = df.drop_duplicates().reset_index(drop=True)

print("Shape after removing rows with missing price and exact duplicates:", df.shape)
print("Remaining missing feature values:", int(df.drop(columns=["price"]).isna().sum().sum()))
```

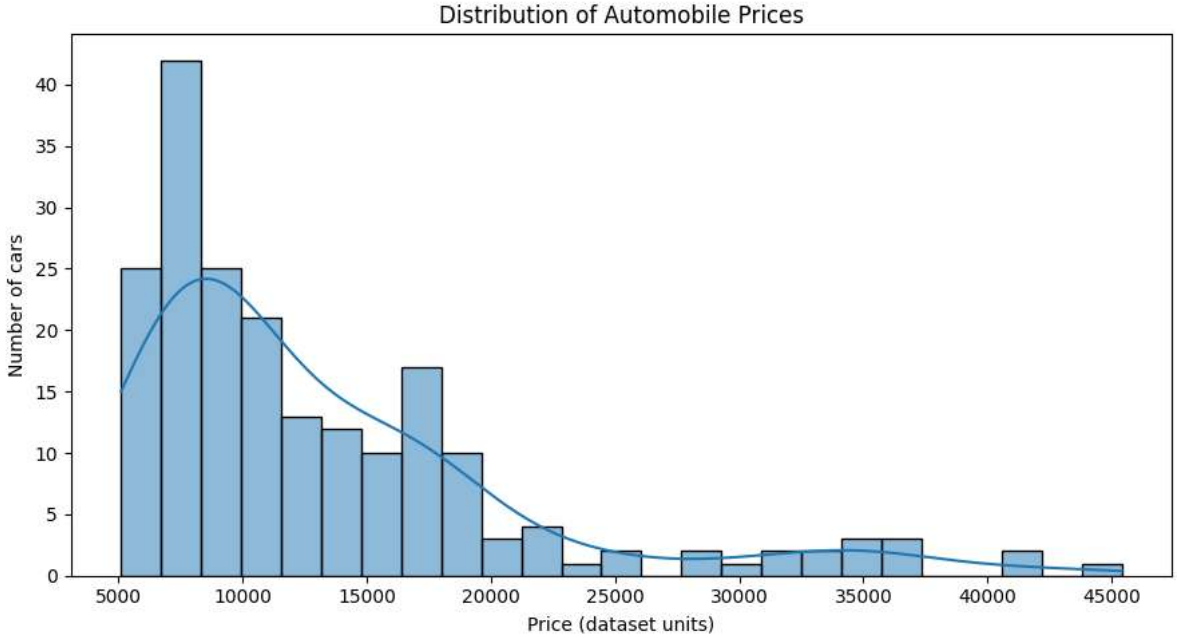
```
Shape after removing rows with missing price and exact duplicates: (201, 26)
Remaining missing feature values: 51
```

5. Exploratory data analysis

5.1 Price distribution

The histogram shows how car prices are distributed. Skewness or unusually expensive vehicles may affect regression error.

```
plt.figure(figsize=(9, 5))
sns.histplot(df["price"], bins=25, kde=True)
plt.title("Distribution of Automobile Prices")
plt.xlabel("Price (dataset units)")
plt.ylabel("Number of cars")
plt.tight_layout()
plt.show()
```

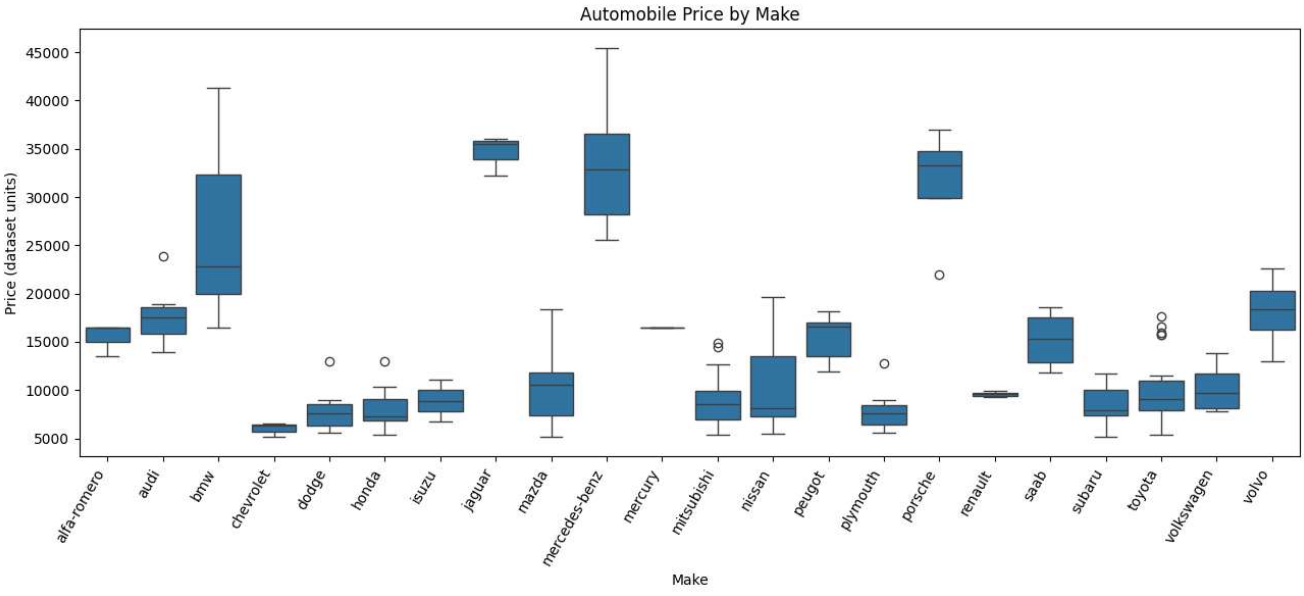


							7500.00	6600.00
city-mpg	205.0	25.219512	6.542142	13.00	19.00	24.00	30.00	49.00
highway-mpg	205.0	30.751220	6.886443	16.00	25.00	30.00	34.00	54.00
price	201.0	13207.129353	7947.066342	5118.00	7775.00	10295.00	16500.00	45400.00

5.2 Price by car make

The box plot compares the distribution of prices for different makes. Categories with few records should be interpreted cautiously.

```
plt.figure(figsize=(13, 6))
sns.boxplot(data=df, x="make", y="price")
plt.title("Automobile Price by Make")
plt.xlabel("Make")
plt.ylabel("Price (dataset units)")
plt.xticks(rotation=60, ha="right")
plt.tight_layout()
plt.show()
```



6. Prepare features and targets

We use the same cleaned dataset for both tasks.

- **Regression target:** `price`, a continuous numerical value.
- **Classification target:** `price_category`, derived from price using fixed thresholds:
  - Budget: price below 10,000
  - Mid-range: 10,000 to below 20,000
  - Premium: 20,000 or higher

The classification model does **not** receive the original `price` column as an input. This prevents direct target leakage. These thresholds are illustrative price bands for this dataset, not universal market definitions.

```
def categorize_price(price):
    if price < 10000:
        return "Budget"
    elif price < 20000:
        return "Mid-range"
    return "Premium"

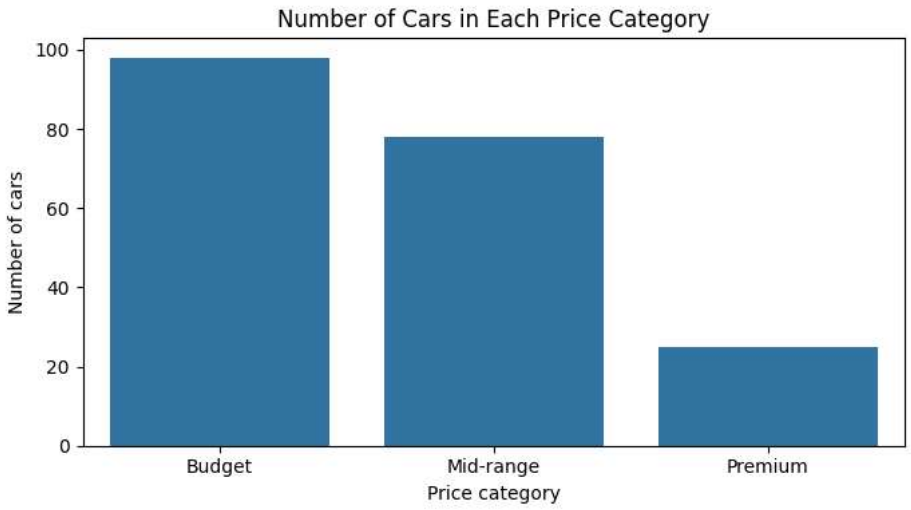
df["price_category"] = df["price"].apply(categorize_price)

print("Classification target distribution:")
display(df["price_category"].value_counts().to_frame("count"))

plt.figure(figsize=(7, 4))
sns.countplot(data=df, x="price_category", order=["Budget", "Mid-range", "Premium"])
plt.title("Number of Cars in Each Price Category")
plt.xlabel("Price category")
plt.ylabel("Number of cars")
plt.tight_layout()
plt.show()
```

```
Classification target distribution:
```

price_category	count
Budget	98
Mid-range	78
Premium	25



7. Build preprocessing pipelines

Numerical columns are imputed with the training-set median and standardized. Categorical columns are imputed with the most frequent training-set value and one-hot encoded.

Using a `Pipeline` and `ColumnTransformer` ensures preprocessing is learned from training data only and applied consistently to test data.

```
# Classification features exclude both target columns.
classification_target = "price_category"
excluded_for_classification = ["price", "price_category"]
X_class = df.drop(columns=excluded_for_classification)
y_class = df[classification_target]

# Regression features exclude price and the category derived from price.
X_reg = df.drop(columns=["price", "price_category"])
y_reg = df["price"]

numeric_features = X_class.select_dtypes(include=["number"]).columns.tolist()
categorical_features = X_class.select_dtypes(exclude=["number"]).columns.tolist()

numeric_preprocessor = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_preprocessor = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("numeric", numeric_preprocessor, numeric_features),
    ("categorical", categorical_preprocessor, categorical_features)
])

print("Numerical features:", len(numeric_features))
print("Categorical features:", len(categorical_features))
print("Feature columns:", X_class.columns.tolist())
```

```
Numerical features: 15
Categorical features: 10
Feature columns: ['symboling', 'normalized-losses', 'make', 'fuel-type', 'aspiration', 'num-of-doors', 'body-style', 'drive-whee
```

8. Classification: predict the price category

We split the data into training and test sets. Stratification attempts to preserve the class proportions in both sets. Logistic Regression is a useful baseline classification algorithm; Random Forest is included as a comparison model.

```
Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    X_class, y_class,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_class
)

classification_models = {
    "Logistic Regression": Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", LogisticRegression(max_iter=3000, class_weight="balanced"))
    ]),
    "Random Forest": Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", RandomForestClassifier(
            n_estimators=300,
            random_state=RANDOM_STATE,
            class_weight="balanced"
        ))
    ])
}

classification_results = []
classification_predictions = {}

for name, model in classification_models.items():
    model.fit(Xc_train, yc_train)
    predictions = model.predict(Xc_test)
```

```
    classification_predictions[name] = predictions

    classification_results.append({
        "Model": name,
        "Accuracy": accuracy_score(yc_test, predictions),
        "Precision (macro)": precision_score(yc_test, predictions, average="macro", zero_division=0),
        "Recall (macro)": recall_score(yc_test, predictions, average="macro", zero_division=0),
        "F1-score (macro)": f1_score(yc_test, predictions, average="macro", zero_division=0)
    })

classification_results_df = pd.DataFrame(classification_results).sort_values(
    "F1-score (macro)", ascending=False
)
display(classification_results_df.round(3))
```

	Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
1	Random Forest	0.902	0.927	0.879	0.898
0	Logistic Regression	0.756	0.719	0.800	0.720

8.1 Classification report and confusion matrix

Accuracy is the proportion of correct predictions. Precision, recall, and F1-score provide additional information about performance across the price categories. The confusion matrix shows which categories are mistaken for one another.

```
best_classification_name = classification_results_df.iloc[0]["Model"]
best_classification_model = classification_models[best_classification_name]
best_classification_predictions = classification_predictions[best_classification_name]

print("Best model by macro F1-score:", best_classification_name)
print("\nClassification report:")
print(classification_report(
    yc_test,
    best_classification_predictions,
    labels=["Budget", "Mid-range", "Premium"],
    zero_division=0
))

cm = confusion_matrix(
    yc_test,
    best_classification_predictions,
    labels=["Budget", "Mid-range", "Premium"]
)

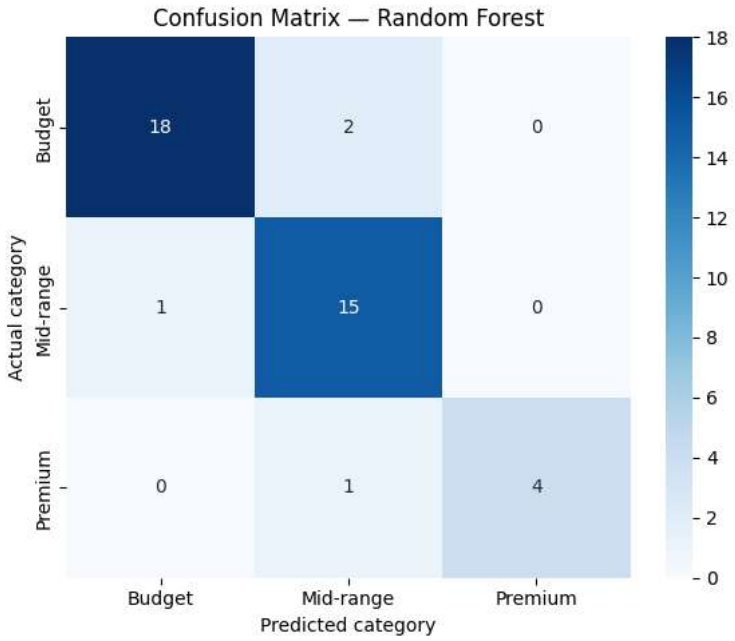
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Budget", "Mid-range", "Premium"],
    yticklabels=["Budget", "Mid-range", "Premium"]
)
plt.title(f"Confusion Matrix – {best_classification_name}")
plt.xlabel("Predicted category")
plt.ylabel("Actual category")
plt.tight_layout()
plt.show()
```

```
Best model by macro F1-score: Random Forest

Classification report:
              precision    recall  f1-score   support

      Budget       0.95      0.90      0.92        20
   Mid-range       0.83      0.94      0.88        16
     Premium       1.00      0.80      0.89         5

    accuracy                           0.90        41
   macro avg       0.93      0.88      0.90        41
weighted avg       0.91      0.90      0.90        41
```



9. Regression: predict the numerical car price

Regression predicts a numerical value instead of a category. We compare Linear Regression with Random Forest Regression. The same feature preprocessing is used, and the `price` target is kept separate from the inputs.

```
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    X_reg, y_reg,
    test_size=0.20,
    random_state=RANDOM_STATE
)

regression_models = {
    "Linear Regression": Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", LinearRegression())
    ]),
    "Random Forest Regressor": Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", RandomForestRegressor(
            n_estimators=300,
            random_state=RANDOM_STATE,
            min_samples_leaf=2
        ))
    ])
}

regression_results = []
regression_predictions = {}

for name, model in regression_models.items():
    model.fit(Xr_train, yr_train)
    predictions = model.predict(Xr_test)
    regression_predictions[name] = predictions

    mse = mean_squared_error(yr_test, predictions)
```

```
    regression_results.append({
        "Model": name,
        "MAE": mean_absolute_error(yr_test, predictions),
        "RMSE": np.sqrt(mse),
        "R2": r2_score(yr_test, predictions)
    })

regression_results_df = pd.DataFrame(regression_results).sort_values(
    "RMSE", ascending=True
)
display(regression_results_df.round(3))
```

	Model	MAE	RMSE	R2
1	Random Forest Regressor	1900.227	2907.204	0.931
0	Linear Regression	1972.147	3252.117	0.914

9.1 Actual versus predicted prices

MAE is the average absolute prediction error, RMSE penalizes larger errors more strongly, and R^2 compares the model with a baseline that predicts the test-set mean. A negative R^2 means the model performed worse than that baseline on this test split.

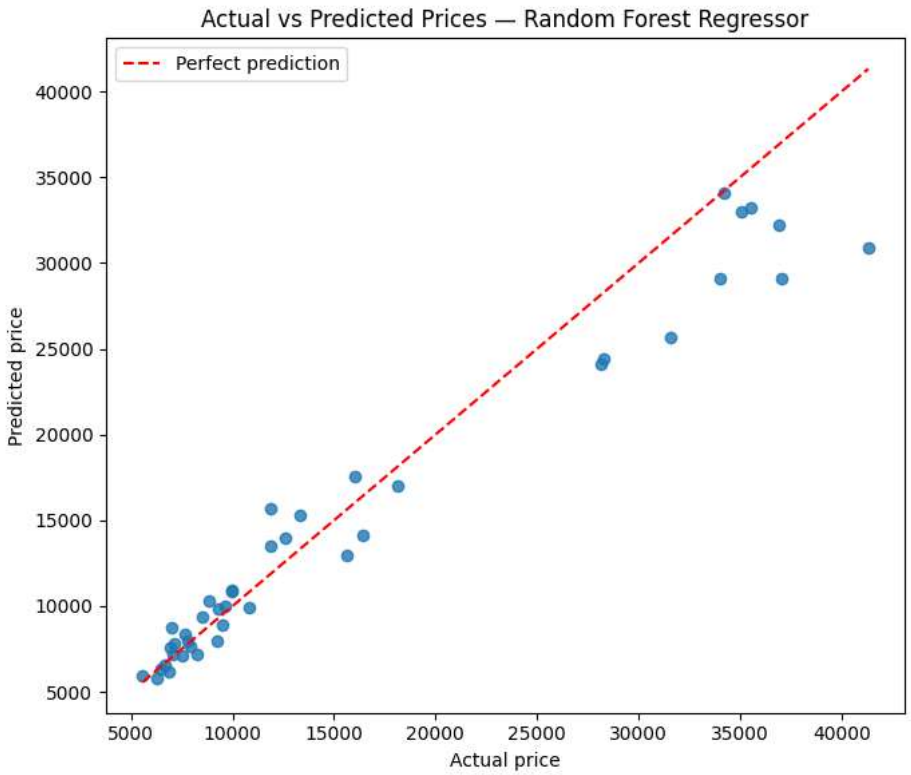
```
best_regression_name = regression_results_df.iloc[0]["Model"]
best_regression_predictions = regression_predictions[best_regression_name]

plt.figure(figsize=(7, 6))
plt.scatter(yr_test, best_regression_predictions, alpha=0.8)

min_value = min(yr_test.min(), best_regression_predictions.min())
max_value = max(yr_test.max(), best_regression_predictions.max())
plt.plot([min_value, max_value], [min_value, max_value], "r--", label="Perfect prediction")

plt.title(f"Actual vs Predicted Prices — {best_regression_name}")
plt.xlabel("Actual price")
plt.ylabel("Predicted price")
plt.legend()
plt.tight_layout()
plt.show()

print("Best regression model by RMSE:", best_regression_name)
```




```
Best regression model by RMSE: Random Forest Regressor
```